BLOG POST

Documenting software architecture with arc42

A brief introduction

August 04, 2022

arc42 is a template for architecture communication and documentation. It is a proven, practical and highly pragmatic approach and takes the pain out of documentation.



DR. GERNOT STARKE

CONTENT

- *Before we dive in...*
- *The structure of arc42*

Like what you saw?

Comments or questions?



arc42 [1] provides everything you ever need to communicate and document your software architecture. It's tool and technology agnostic, therefore you can use arc42 for arbitrary systems. arc42 is open source and completely free to use.

This article provides a quick overview of arc42 and its parts. For more information, please consult the (rather extensive) documentation [2] available on their documentation website (which, since V8/January 2022, contains several practical examples).

arc42 answers the following two questions in a pragmatic way and can be tailored to your specific needs:

- *What* should you document/communicate about your architecture?
- *How* should you document/communicate?

If you work under time or resource constraints, you will most likely skip certain parts of arc42. Our upcoming post on "lean documentation" will give guidance for such cases.

Before we dive in...

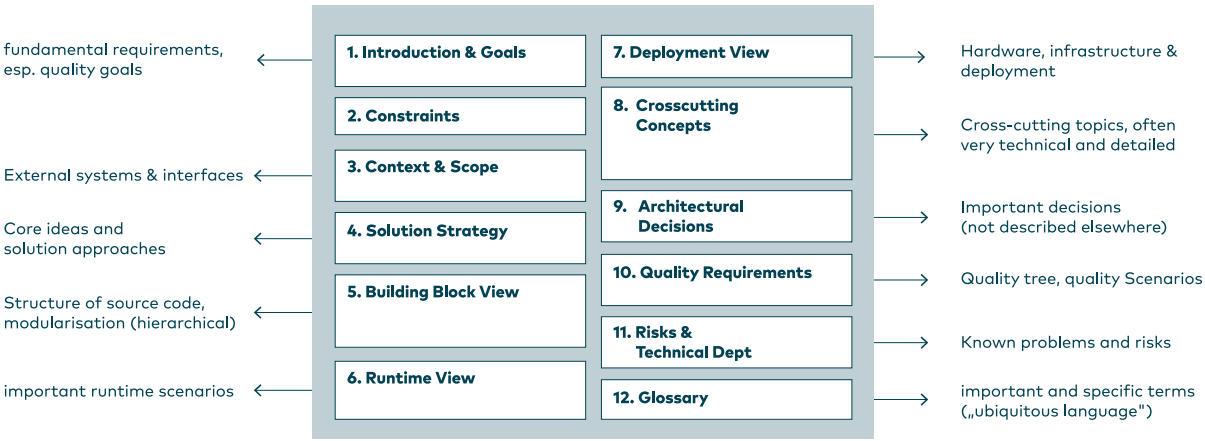
- Everything is optional - there is **no** need to fill in every section of the template. Treat arc42 documentation like the compartments of a cabinet, similar to the image below: The cabinet has a value, even if certain compartments remain empty.



- The order of these compartments is optimized for **reading** and understandability. You create your content in any order you like or your project requires. For example, in many development projects, the context (section 3) is defined and created first, followed by the top quality requirements (section 1.2).
- arc42 is completely tool agnostic. You might create and maintain arc42 in your tool chain of choice: A wiki, a collection of Office documents, plain Markdown or Asciidoc, one of the commercial UML tools or even LaTeX. Whatever your tool of choice is, arc42 has you covered.

The structure of arc42

arc42 proposes a dozen sections (aka compartments) for your documentation. These are organized top-down to improve understandability.



1. Introduction and goals

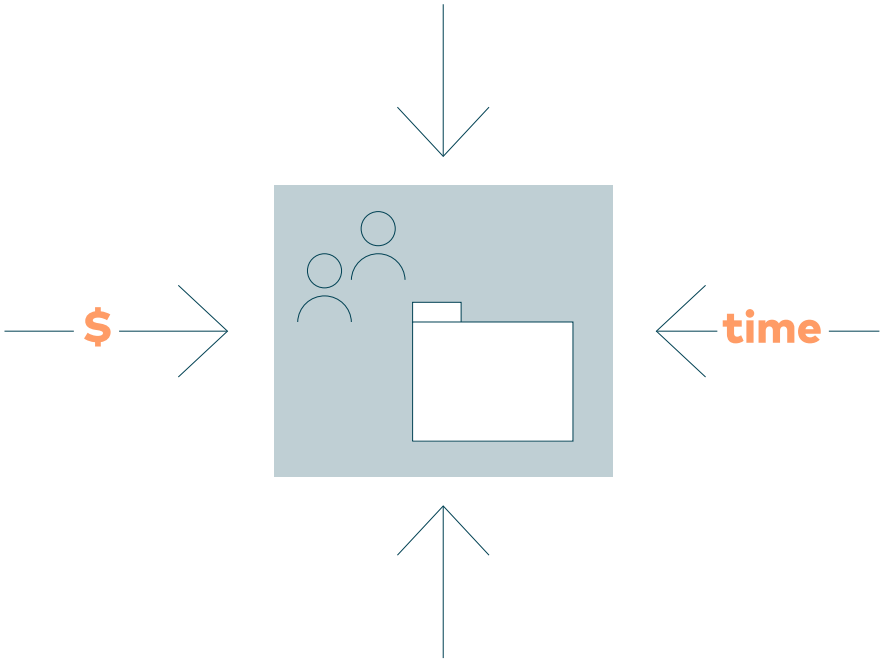
Short description of the requirements and driving forces of your system: You should provide an extract or abstract of requirements. Restrict yourself to the top three (max five) **quality requirements** for the architecture, which have the highest priority for the major stakeholders.

Include a table of important **stakeholders** with their expectation regarding architecture or documentation.



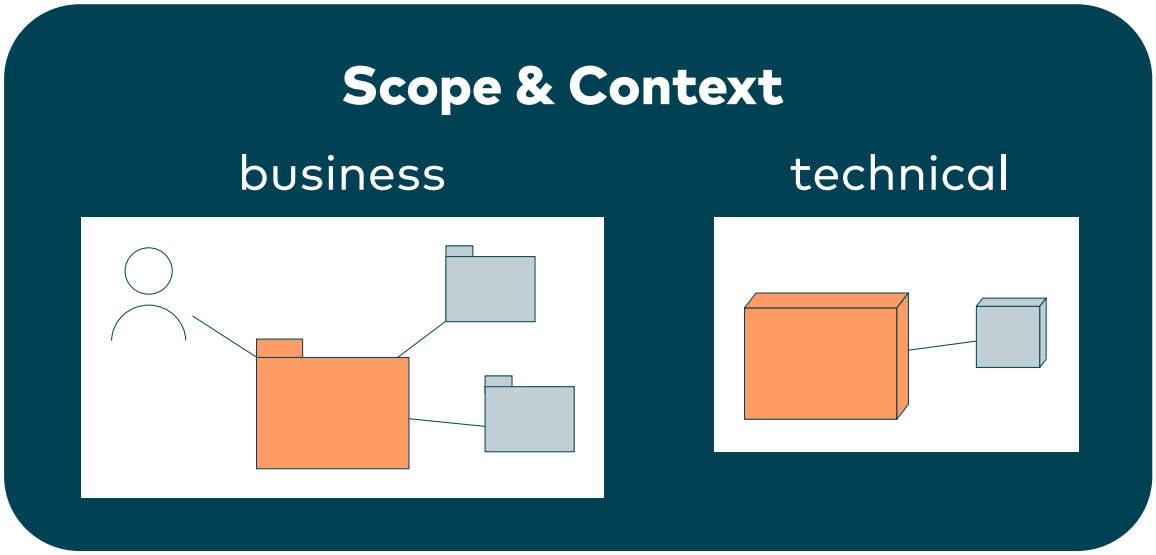
2. Constraints

Anything that constrains teams in design and implementation decisions, or decision about related processes. Constraints can sometimes go beyond individual systems and might valid for whole organizations and companies (e.g. company-wide technology choices or government regulations). Time and money are common constraints in many organizations.



3. Context and scope

The context delimits your system from its (external) communication partners (neighboring systems and users). It specifies or documents the **external interfaces**. You should always document the context from a business or domain perspective. If infrastructure or specific hardware plays an important role, you might also show a technical perspective.

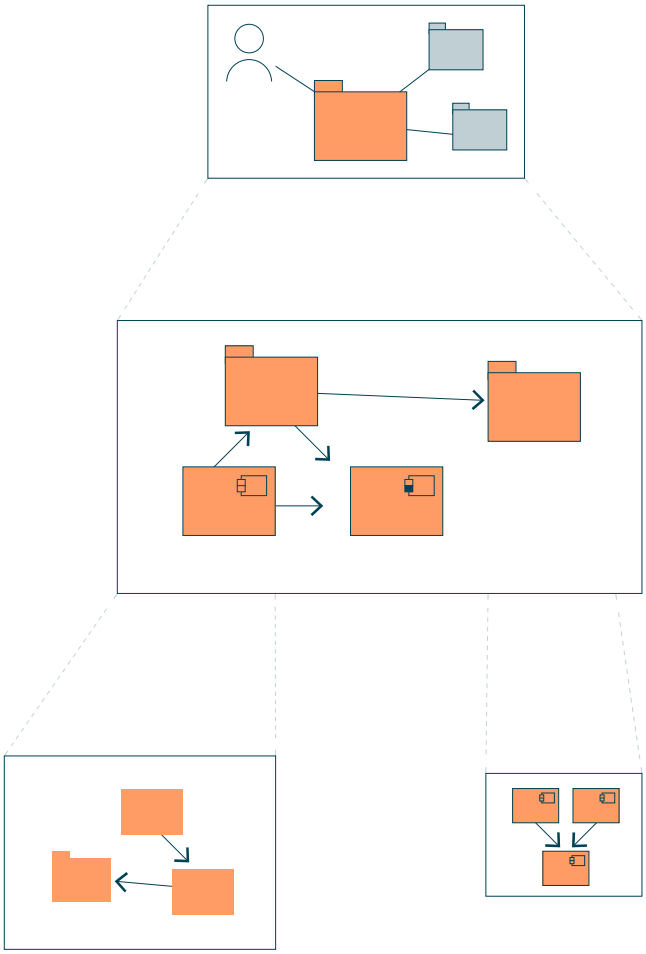


4. Solution strategy

Summary of the fundamental decisions and solution strategies that shape the architecture: can include technology, top-level decomposition, approaches to achieve top quality goals and relevant organizational decisions.

5. Building block view

Usually, boxes and arrows, showing the high-level code structure of the system. To phrase it a little more formal: The building block view explains the static structure of the system and contains, abstractions of source-code. It refines the context view, where the complete system is depicted as a black box.



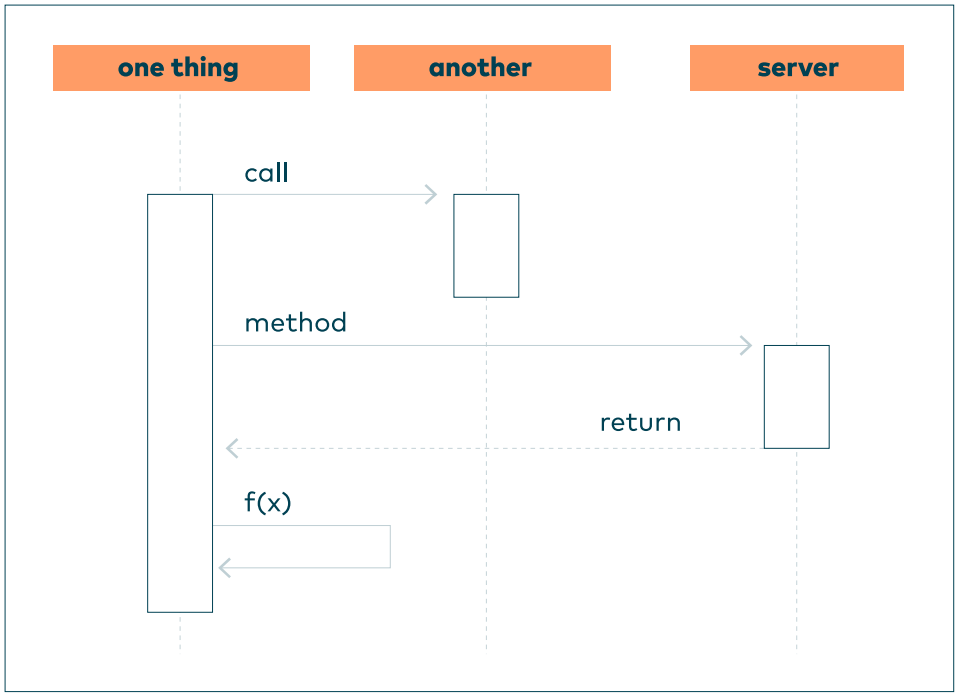
- Level 1 contains the major subsystems, components or parts of the system. In some cases, this coarse overview provides enough structural overview.
- Level 2 refines one or more elements from level 1: Create a separate white box (plus explanations) for every level-1 building block you would like to explain in detail.

You can drill down to an arbitrary level of detail, but keep in mind the following:

Recommendation *Try to stick to level-1, as it often gives enough guidance and understanding for most stakeholders.*

6. Runtime view

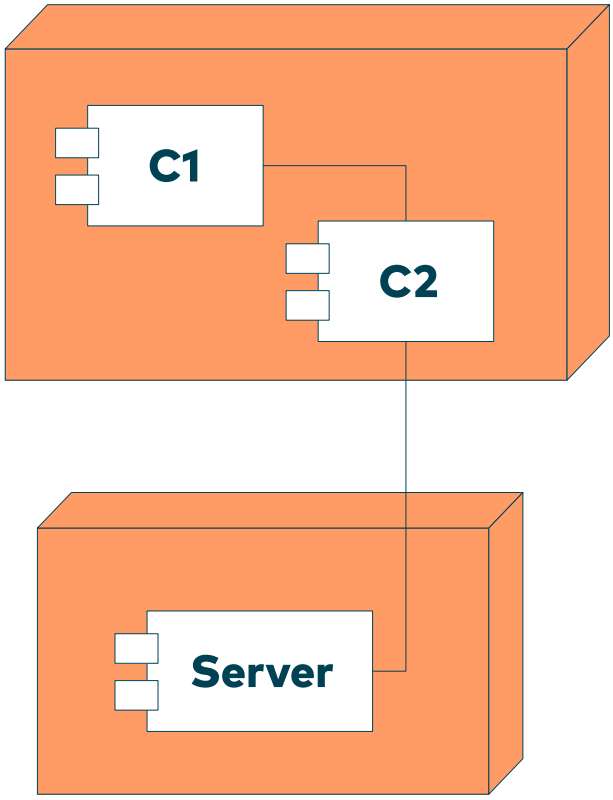
The runtime view explains the behavior or processing of one or several building blocks. It serves as companion to the static building block view from section 5 above. The runtime view might explain important use cases or features, interactions at critical external interfaces, error and exception behavior.



Recommendation *Use runtime scenarios primarily during development to validate or design building block structures. They help to identify missing or unnecessary dependencies. Keep only a small number of scenarios in your documentation.*

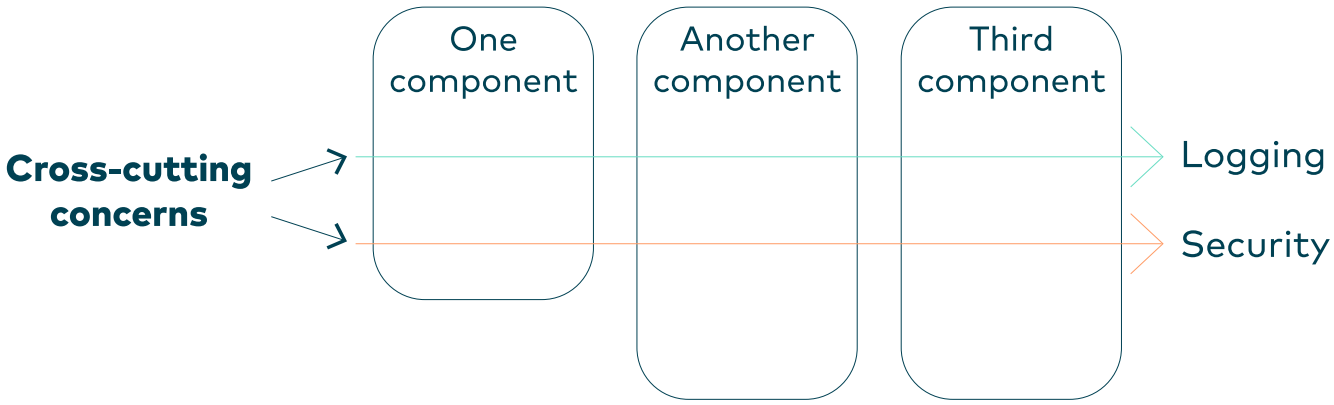
7. Deployment view

Software needs hardware to execute on, that's where the deployment view comes into play: It shows the technical infrastructure with environments, computers, processors, networks and network-topologies. In addition, it maps the software building blocks to those infrastructure elements.



8. Cross-cutting concepts

Overall, principal regulations and solution approaches relevant in multiple parts (→ cross-cutting) of the system. Concepts are often related to **multiple building blocks**. Include different topics like domain models, architecture patterns and -styles, rules for using specific technology and implementation rules.



9. Architecture decisions

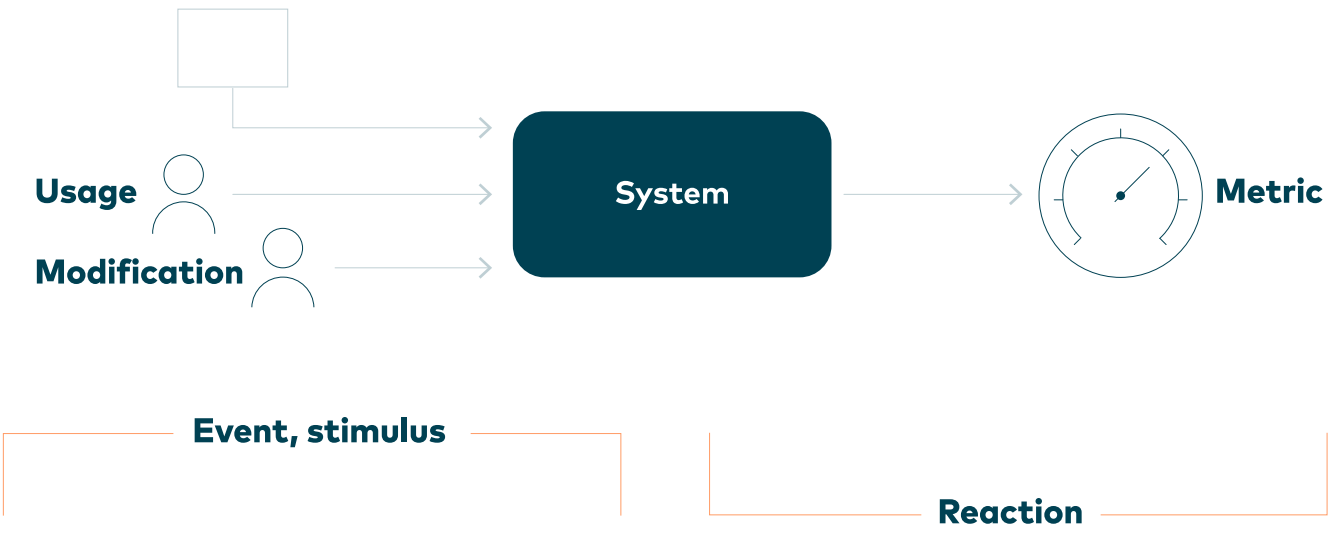
Keep a collection of *architecturally significant* decisions that are important, expensive, critical, large scale or risky including rationales, that are not recorded elsewhere.

Please use your judgement to decide whether an architectural decision should be documented here or whether you better document it locally (e.g. within the building block view or any cross-cutting concept). You may have already captured the most important decisions of your architecture in the solution strategy.

Recommendation: *Use Architecture Decision Records to capture decisions in a simple, structured yet pragmatic way.*

10. Quality requirements

Quality requirements, often described as scenarios. You may use a quality tree to provide a high-level overview. The most important quality goals should have been already described in section 1.2. (quality).



As the definition of quality scenarios is very abstract, let's look at some examples:

Topic	Scenario
Performance	System shall perform all checks on a 100MByte file in less than 10 seconds.
Operability	The temperature range in which the correct functioning of the device is ensured should range from -25 degrees to +85 degrees

Topic	Scenario
Maintainability	A new additional verification algorithm can be included in less than one day.

11. Risks and technical debt

Known technical risks or technical debt. What potential problems exist within or around the system? What does the development team feel miserable about.

In case somebody reviewed or audited your system, their findings might be included here.

12. Glossary

Important domain and technical terms that stakeholders use when discussing the system. Please include **only specific terms** - and avoid explaining REST, HTTPS or other words that have common explanations.

In addition, the glossary can serve as the translation reference if you work in a multi-language (*international*) environment.

Term	Definition
aarkward	....
churizzo	.....
domoklesian	....
growidarian	.....
klicktilazation	....

Like what you saw?

Give it a try! Document a few of your architecture decisions, sketch your scope and context and explain a few key technical concepts - and you have your first version of your architecture documentation.

Download the template in a format you like. Unpack and start documenting. In case you have questions or want to see examples, consult the extensive documentation site [2].

In case you want to know more, we have you covered:

- I wrote about the principles of technical documentation. That deep-dive article explains the fundamental requirements, your technical documentation needs to fulfill - and many proven approaches how to achieve these.
- Another article on lean documentation is in the making, please stay tuned. Until then, watch the video "Sparsame Dokumentation" (oops, sorry, it's in German)

Comments or questions?

Leave a comment right below - or open an issue on the arc42 site. Otherwise, thanx for reading this article.

Image of the cabinet by Jan Kolar from Unsplash

1 https://arc42.org. The template has been created by Dr. Peter Hruschka and Dr. Gernot Starke. They published the first version in 2005, after having used predecessor versions for many years in different large and medium-sized projects. ↩

2 https://docs.arc42.org. The extensive documentation website, containing more than 140 tips and 30 examples on how to use arc42. ↩

TAGS

Documenting Software Architecture with arc42 Software Architecture



DR. GERNOT STARKE

Fellow

As an INNOQ fellow, Gernot participates in the strategic development of the company's consulting and implementation products. He supports clients as a consultant for software architecture in general and documentation in particular.

RECOMMENDED

ARTICLE

Grundlagen der Softwarearchitektur: Teil 3

In dieser Folge der Mini-Serie klären wir, was Sie alles für Softwarearchitektur tun müssen. Zuerst schauen wir auf die notwendigen Aufgaben und Tätigkeiten. Anschließend diskutieren wir einige der notwendigen Kompetenzen und Fähigkeiten für die Architekturaufgabe.



DR. GERNOT STARKE

ARTICLE

Principles of technical documentation

This article collects fundamental requirements for technical documentation, especially software architecture documentation, together with ideas how to satisfy those.



DR. GERNOT STARKE

ARTICLE

Infrastructure in architectural documentation



ANJA KAMMER



STEFAN ZÖRNER

TALKS

TALK

16 JUN 2026

Code wird billig, Architektur entscheidend.

iSAQB® Software Architecture Forum 2026 | 09:15 - 10:00



DR. GERNOT STARKE

TALK

16 JUN 2026

„Garbage-In“ ist kein Schicksal: Bessere Anforderungen für Entwicklungsteams

iSAQB® Software Architecture Forum 2026 | 11:15 - 12:15



DR. GERNOT STARKE

TALK

30 JUN 2026

Die Canvas-Kette: Orientierung im Projekt-Dschungel

DWX 2026 | 14:15 - 15:15



DR. GERNOT STARKE

TALK

30 2026
JUN

Cognitive Biases im Software Engineering: Warum gute Architekt:innen schlechte Entscheidungen treffen

DWX 2026 | 17:00 - 18:00



DR. GERNOT STARKE

Get in touch

Use our contact form or send us an email to info@innoq.com. You can send us encrypted emails, too. Feel free to use our S/MIME certificates (.cer, .p7b, .pem).

Name*

Email*

z.B. name@example.com

Role

Company

Message*

Anti-Robot Verification

Click to start verification

FriendlyCaptcha

Privacy Notice:
Your personal data will be processed exclusively for the purpose of handling your inquiry and any follow-up questions. The legal basis for processing is Art. 6 Para. 1 lit. f GDPR. For more information, please refer to our Privacy Policy .

Send

Offices

innoQ Deutschland GmbH

Krischerstr. 100
40789 Monheim am Rhein
Tel (+49) 2173 3366 0

DIRECTIONS

Ohlauer Str. 43
10999 Berlin

DIRECTIONS

Wendenstraße 130
20537 Hamburg

DIRECTIONS

Spichernstraße 44
50672 Köln

DIRECTIONS

Ludwigstr. 180 E
63067 Offenbach

DIRECTIONS

Kreuzstr. 16
80331 München

DIRECTIONS

innoQ Schweiz GmbH

Schutzengelstr. 57
6340 Baar
Tel (+41) 41 743 01 11

DIRECTIONS

Hardturmstrasse 253
8005 Zürich

DIRECTIONS

Links

Blog & Articles

Talks

Podcasts

Technology Lunch

Technology Briefing

Trainings

Meetups

Primers

Books

Inside Technology: The Book

Management

Contact

Privacy

Whistleblower

Legal Notice

Services

Architecture Strategy

Software Architecture and Development

Data & AI

IT Security

Legacy Modernization

Software Reviews

Digital Platforms and Infrastructures

Knowledge Transfer, Coaching and Trainings

Digital Product Development

Our Newsletters

Never miss out on interesting articles, events and podcasts on architecture, development and technology trends!

Right now, our newsletters are only available in German.

☐ **Der INNOQ Newsletter**
Exciting articles, event tips and podcasts on architecture, development and technology trends. Published approx. once per month. (German)

☐ **Das Digitale Rauchzeichen**
Success stories from digitalization and modernization from INNOQ Switzerland. Published up to 4 times per year. (German)

Email

First name

Last name

Subscribe

Find us on